

Table of Contents

Table of Contents.....	1
1. Introduction: The Motivation and Philosophy of HOG.....	2
2. Intuition Before Math.....	2
HOG Pipeline Overview.....	2
Core Intuitions.....	3
Visual Intuition: From Pixels to Hog.....	3
3. Technical Foundations.....	4
3.1 Gradient Computation.....	4
3.2 Cell Histograms : Spatial Binning.....	4
Why Cells? (Noise vs. Structure).....	4
Orientation Binning.....	5
The Voting Process.....	5
3.3 Block Normalisation.....	5
The Solution.....	5
The L2-Norm.....	5
Overlapping Blocks.....	6
3.4 Feature Vector Construction.....	6
The Sliding Window.....	6
The Dalal & Triggs Calculation.....	6
4. Worked Numerical Example: HOG in Action.....	7
4.1 The Setup: A Simplified Patch.....	7
5. Parameter Choices.....	9
5.1 Cell Size.....	9
5.2 Number of Orientation Bins.....	9
5.3 Block Size and Overlap.....	10
5.4 Window Size.....	10
5.5 Summary of Parameter Trade-offs.....	10
Effects of Extreme Choices.....	11
6. Failure Modes.....	11
6.1 Rotation Sensitivity.....	11
6.2 Scale Sensitivity.....	12
6.3 Occlusion and Articulation.....	12
6.4 Background Clutter and the “HOGgles” Effect.....	12
6.5 Lighting Extremes.....	12
6.6 Lack of Colour and Texture Information.....	13
6.7 Summary of HOG Limitations.....	13
Quick Reference.....	14
Key Concept.....	14
Equations / Functions.....	14
Parameter Ranges.....	14
Glossary of Common Misconceptions.....	15
Concise Definitions of Key Terminology.....	15

1. Introduction: The Motivation and Philosophy of HOG

Imagine trying to recognise someone in a crowd by their silhouette rather than the exact colours of their clothes. Even if the lighting changes or they wear something different, their overall shape usually remains recognisable. The fine details, colours, shadows, brightness, and texture, may change dramatically, but the outline of the person stays relatively consistent.

This idea forms the foundation of Histogram of Oriented Gradients (HOG). To a computer, an image is just a grid of pixel values with no inherent meaning, and raw pixels are highly unstable. Small changes in lighting, shadows, noise, position, or scale can drastically alter them even when the object stays the same (e.g. a person in sunlight vs shadow). A good feature should therefore discard irrelevant variations like colour, shadows, or accessories, while preserving the information that distinguishes one object from another [2].

HOG addresses this problem by focusing on structure rather than exact appearance. Instead of storing every pixel, it examines where pixel intensities change sharply i.e the gradients that form edges, contours, and outlines. These gradients usually remain far more consistent under changing conditions, making them a more robust representation of an object's shape. By capturing the distribution of edge directions across an image, HOG extracts the essential structural information while discarding unstable details such as colour variations, shadows, and noise.

To further improve robustness, HOG applies block normalisation, which reduces sensitivity to illumination and contrast changes. The result is a simplified but reliable representation that preserves the important shape information needed for tasks such as object detection and image classification.

2. Intuition Before Math

HOG Pipeline Overview

Before diving into the mathematics, it is useful to view HOG as a step-by-step pipeline that transforms raw pixels into a compact representation of image structure through gradient extraction, aggregation, and normalization.

1. Pre-processing: Normalizing gamma and color to reduce the impact of extreme lighting shadows.
2. Gradient Computation: Calculating the horizontal (G_x) and vertical (G_y) changes for every pixel to identify edges.
3. Spatial / Orientation Binning (Cells): Grouping pixels into 8×8 "cells" and casting "votes" into a histogram based on the direction of their gradients.
4. Block Normalization: Grouping cells into larger overlapping "blocks" and normalizing the vectors to make them robust to lighting variations.
5. Descriptor Concatenation: Flattening all normalized blocks into one final, long feature vector ready for use.

Core Intuitions

Why Gradients Capture Shape

Gradients capture shape because object boundaries and contours appear where pixel intensities change sharply. These regions produce large gradient magnitudes, making gradients effective for detecting edges and silhouette information, while flat/uniform regions contribute little useful structure. By focusing on gradients instead of raw pixel values, HOG emphasises the boundaries that define object shape while reducing sensitivity to absolute brightness changes.

Why Orientation Matters

Orientation matters because gradients are defined by both magnitude and direction. The orientation describes the direction of strongest intensity change, which corresponds to local edge directions. Different objects produce distinctive orientation patterns. For example, a standing person creates mostly vertical edges along the body and horizontal edges at the shoulders.

To make these patterns robust, HOG groups nearby pixels into local cells and stores the distribution of edge orientations within each cell. This reduces sensitivity to noise and small movements while preserving the object's overall structure.

Visual Intuition: From Pixels to Hog

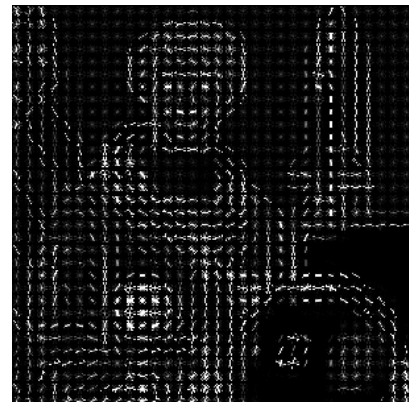
To further understand HOG, compare the following three views of the same image:

Pixel View (see right): A chaotic grid of numbers (0–255) that changes completely if you turn up the brightness.

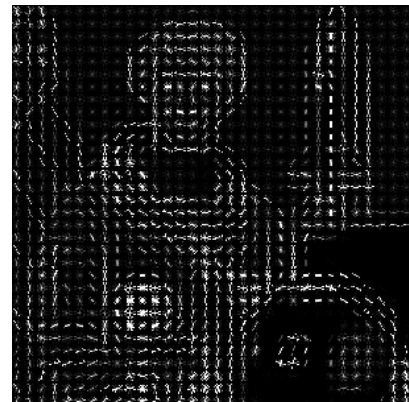
33	37	40	42	44	47	49	49	49	48
56	59	51	51	50	47	44	39	34	31
34	34	26	27	27	26	23	19	15	12
10	21	24	20	14	11	11	13	15	15
11	23	14	13	12	11	11	14	18	21
12	23	8	12	14	13	11	12	16	21
14	18	13	18	22	18	11	7	10	15
14	10	17	22	24	20	12	7	8	12
11	5	13	16	18	16	11	9	10	13
7	6	9	9	9	9	8	8	9	10



Gradient View (see left): A "glow" around the edges of the object, showing where the shape is, but losing the specific direction of the lines.



HOG View (see right): A grid of star-shaped icons. Each star represents a cell, and the length of each spoke shows how many edges in that cell are pointing in that specific direction. This is the structural fingerprint the computer sees.



3. Technical Foundations

This section details the four-stage technical pipeline that converts a standard image into a HOG feature vector.

3.1 Gradient Computation

The first step in HOG is detecting where image intensity changes using discrete derivatives. For each pixel, the horizontal gradient G_x and vertical gradient G_y are computed using simple filters such as $[-1,0,1]$ and its transpose. (G_x) measures horizontal intensity change, while (G_y) measures vertical change. Small gradients indicate flat regions, whereas large gradients usually correspond to edges and object boundaries.

The below example demonstrates how we use the $[-1,0,1]$ filter to calculate G_x :

80	120	50
----	-----	----

$$G_x = (-1)(80) + (0)(120) + (+1)(50) = -30$$

The logic for calculating G_y is analogous.

Magnitude and Orientation

Once G_x and G_y are calculated, HOG uses them to find two important values:

1. Magnitude: how strong the edge is

$$\sqrt{G_x^2 + G_y^2} \quad [4]$$

2. Orientation : which direction the gradient points (the angle of the edge)

$$\arctan \frac{G_y}{G_x} \quad [4]$$

In HOG, we typically use unsigned gradients, meaning the angles are mapped between 0° and 180° rather than 360° . This is because the shape of an edge is the same whether it transitions from black-to-white or white-to-black.

3.2 Cell Histograms : Spatial Binning

After computing gradients, the representation is still too large and noise sensitive, while treating the whole image globally would lose spatial detail. HOG addresses this by dividing the image into small local regions called cells. Dalal & Triggs found that cell sizes of about 6–8 pixels work well for human detection, regardless of block size [1].

Why Cells? (Noise vs. Structure)

Cells provide a balance: they summarize local edge patterns while still preserving where those patterns occur. In this way we gain spatial invariance meaning if an object moves by a few pixels, its pixel-level gradients change locations, but the sum of those gradients within the cell remains roughly the same.

Orientation Binning

Inside each cell, we create a histogram of the orientations. The standard HOG uses 9 bins, with each bin covering 20° (totaling 180°).

0° 20° 40° 60° 80° 100° 120° 140° 160°

The Voting Process

Every pixel in the 8×8 cell votes into one of these 9 bins.

The vote is weighted by its gradient's magnitude. This means strong edges contribute more than weak edges. In other words, a strong edge gets a larger vote into the histogram.

HOG uses bilinear interpolation to distribute a gradient's vote between nearby orientation bins. If an angle lies between two bins, its contribution is split proportionally based on distance. For example, 25° is shared between the 20° and 40° bins, with more weight given to 20° . This produces a smoother, more robust descriptor that is less sensitive to small shifts or noise.

In some HOG implementations, interpolation can also be applied spatially so pixels contribute to neighbouring cells. This reduces aliasing and makes the descriptor less sensitive to small shifts, preventing abrupt changes when gradients move between cells.

3.3 Block Normalisation

Cell histograms capture local edge structure, but they are still sensitive to lighting changes. The same object may produce much larger gradient magnitudes under bright illumination than in darker conditions, causing HOG to treat identical shapes differently unless this variation is normalised.

The Solution

The solution is block normalisation, where neighbouring cells are grouped into larger blocks (commonly 2×2 cells) and their histograms are normalised so the descriptor depends more on edge patterns than absolute brightness or contrast. Since gradients are based on pixel differences, adding a constant brightness to all pixels does not change them, making the descriptor invariant to illumination changes. [4]

For example:

Dark image: edge values = [2, 4, 6]

Bright image: edge values = [4, 8, 12]

The second image has larger values, but the pattern is the same. Normalisation makes these two representations much more similar.

The L2-Norm

A common normalisation method is L2 normalisation, which divides the block vector by its overall length:

$$v_{norm} = \frac{v}{\sqrt{\|v\|^2 + \epsilon^2}}$$

Dalal and Triggs also used L2-Hys normalisation [1]. Which simply follows up L2 Normalisation with an extra clipping and renormalising step. The clipping step prevents extremely strong edges from dominating the descriptor.

Overlapping Blocks

Another important detail is that blocks usually overlap. This means the same cell can appear in multiple blocks. At first, this seems redundant, but it improves robustness because the same local edge pattern is normalised against different neighbouring contexts.

Overlapping blocks increase the size of the final feature vector and require more computation, but they usually improve detection accuracy.

3.4 Feature Vector Construction

After gradients, cells, histograms, and blocks have been computed, HOG constructs the final descriptor.

The Sliding Window

A fixed-size window moves across the image, and for each position, we calculate the HOG descriptor. This was common in classical computer vision systems, including Viola and Jones' real-time face detector, where many candidate image windows were evaluated using simple features and a cascade classifier [3].

- For object detection, this allows the model to ask: "Does this window contain the object I am looking for?"
- Similarly, in pedestrian detection, the window might slide across the image looking for regions whose HOG features resemble the shape of a person.

Inside each detection window, all normalised block histograms are flattened and concatenated into one long 1D feature vector. So instead of storing an image as pixels, HOG stores it as a numerical summary of local edge structure.

The Dalal & Triggs Calculation

Dalal and Triggs used a classic pedestrian detection window of 64×128 pixels. This size was chosen because it roughly matches the aspect ratio of an upright pedestrian [1].

Using standard parameters such as 8×8 cells 2×2 blocks with a stride of 8 pixels:

- Cells: There are 8x16 = 128 cells.
- Blocks: Because the blocks overlap and slide with an 8-pixel stride, we have (8-1) horizontal positions and (16-1) vertical positions.
 - 7x15 = 105 blocks.
- Features per Block: Each block has 4 cells, and each cell has 9 bins.
 - 4x9 = 36 values per block.
- Final Vector Length:
 - 105 blocks x 36 values = 3,780 features

That 3780-dimensional vector is then passed into a classifier, such as a Support Vector Machine, which learns to distinguish between object and non-object windows [2].

4. Worked Numerical Example: HOG in Action

This section shows a simplified numerical example of how HOG works. The goal is not to calculate a full real-world HOG descriptor, but to understand the logic behind each step:

1. Compute gradients
2. Compute magnitude and orientation
3. Assign votes to orientation bins
4. Form a cell histogram
5. Normalize the block

We use a small image patch and focus on one pixel for simplicity. In full HOG, steps 1–2 are applied to all pixels in each cell, steps 3–4 to each cell, and step 5 after all cell histograms are computed.

4.1 The Setup: A Simplified Patch

Imagine we have a small 3x3 grayscale image patch. We are focusing on the center pixel (highlighted in bold) to see how its HOG features are calculated.

	x0	x1	x2
y0	80	100	120
y1	80	100	190
y2	80	142	120

Step 1: Compute Gradients (Gx, Gy)

We use the simple 1-D centered derivative kernels: $[-1, 0, 1]$ and $[-1, 0, 1]^T$.

- Horizontal Gradient (Gx): Subtract the pixel to the left from the pixel to the right.
 - $G_x = 190 - 80 = 110$
- Vertical Gradient (Gy): Subtract the pixel above from the pixel below.
 - $G_y = 142 - 100 = 42$

Step 2: Compute Magnitude and Orientation

Now, we convert these two directional changes into a single vector with a "strength" and an "angle."

1. Magnitude (M): Using the Pythagorean theorem.

$$M = \sqrt{110^2 + 42^2} = \sqrt{12100 + 1764} = \sqrt{13864} \approx \mathbf{117.7}$$

2. Orientation (theta): Using the arc tangent function.

$$\theta = \arctan\left(\frac{42}{110}\right) \approx \mathbf{20.9^\circ}$$

Step 3: Assign Votes to Bins (With Interpolation)

We are using the standard 9-bin histogram (0° , 20° , 40° , ..., 160°). Our calculated orientation is 20.9° .

This value falls between the 20° bin and the 40° bin. To prevent sudden jumps in data when an angle changes slightly, we split the magnitude (117.7) between these two neighbors based on how close the angle is to each.

The "20.9° Split" Calculation:

- Distance to Bin 20: $20.9 - 20 = 0.9$
- Distance to Bin 40: $40 - 20.9 = 19.1$
- Total Bin Width: 20

Weight for Bin 40 (the "far" bin):

$$\text{Vote}_{40} = 117.7 \times \left(\frac{0.9}{20}\right) \approx \mathbf{5.3}$$

Weight for Bin 20 (the "near" bin):

$$\text{Vote}_{20} = 117.7 \times \left(\frac{19.1}{20}\right) \approx \mathbf{112.4}$$

Intuition: Because 20.9° is very close to 20° , Bin 20 gets the vast majority of the "strength" (112.4), while Bin 40 gets a tinier part of the vote (5.3).

Step 4: Form the Cell Histogram

In a real HOG implementation, we would repeat the steps above for every pixel in the cell. We then sum all the votes.

Imagine that after processing all 64 pixels in our cell, we get the following 9-bin histogram (a 9-dimensional vector):

$$H_{\text{cell}} = [15, \mathbf{250}, 40, 10, 5, 5, 10, 20, 15]$$

(Note the peak at 250 in the 20° bin, reflecting the strong edge we just calculated).

Step 5: Block Normalization

Finally, we must normalize. Let's assume this cell is part of a 2x2 block. This block would contain four cell histograms concatenated together.

For simplicity, let's assume all four cells in our block are identical to the one we just calculated. Our non-normalized block vector (v) would be:

$$v = [H_1, H_2, H_3, H_4] \quad (\text{a vector of } 9 \times 4 = 36 \text{ values})$$

To normalize using the L2-norm, we calculate the "length" of this 36-dimensional vector:

1. Square every value: $15^2, 250^2 \dots$
2. Sum them all up: Let's say the sum of squares is 500,000.
3. Square root: $\sqrt{500,000} \approx 707.1$.
4. Divide: Divide every single value in the 36-bin vector by 707.1.

Result: The final values in our feature vector will now all be between 0 and 1. If we suddenly doubled the brightness of the image, the raw gradient magnitudes would double, but the ratio between them would stay the same, and this normalization step would bring them back to these same decimals. This is why HOG is robust to lighting!

Summary of the Numerical Flow

1. Pixels -> Gradients ($G_x=110, G_y=42$)
2. Gradients -> Polar Coordinates ($M=117.7, \theta=20.9^\circ$)
3. Polar -> Histogram Votes (112.4 into Bin 20, 5.3 into Bin 40)
4. Cell Votes -> Block Vector (Concatenate 4 cells into 36 values)
5. Block Vector -> Normalized Descriptor (Divide by total magnitude)

5. Parameter Choices

HOG performance depends strongly on parameter tuning, as choices like spatial and orientation binning affect accuracy, robustness, and computation cost. Dalal & Triggs show that even small changes can significantly impact results, making HOG a balance between detail, robustness, and efficiency [1].

5.1 Cell Size

A cell is the basic unit in HOG where pixel gradients are summarised into orientation histograms. Instead of using noisy individual pixels, gradients are grouped to capture local edge structure. Cell size balances detail and robustness.

Small cells (e.g. 4×4) capture fine detail but are more sensitive to noise and positional changes, increasing feature size. Large cells (e.g. 16×16) are more stable and invariant but lose spatial detail by averaging over larger regions.

A common choice is 8×8 pixels, which provides a good balance between local detail, robustness, and computational efficiency.

5.2 Number of Orientation Bins

The number of orientation bins controls how finely HOG encodes edge directions. A common choice is 9 unsigned bins over 0° – 180° ($\sim 20^{\circ}$ each), which captures key orientations while remaining robust to small angle changes.

Too few bins reduce sensitivity and merge distinct edge directions, while too many make the histogram sparse and noise-sensitive. Dalal & Triggs found that more than 9 bins gives little improvement for object detection [1].

HOG uses unsigned gradients (0° – 180°) because it focuses on edge orientation, not brightness direction. Dark-to-light and light-to-dark edges represent the same boundary, so they are treated equally. Signed gradients add polarity sensitivity with little benefit for detection.

5.3 Block Size and Overlap

After computing cell histograms, HOG groups neighbouring cells into blocks for normalisation, typically 2×2 cells. With 9 bins per cell, each block has 36 values. This step reduces the effect of illumination by focusing on relative edge patterns rather than absolute brightness.

Smaller blocks preserve local detail, while larger blocks improve robustness to lighting but may lose precision. Blocks usually overlap (stride of one cell), so each cell contributes to multiple blocks, improving stability but increasing feature size and computation. Overall, block design balances robustness, detail, and efficiency.

5.4 Window Size

The detection window in HOG is a fixed region used to extract a feature vector for classification. Dalal & Triggs used a 64×128 window for pedestrians [1]. The 2 to 1 aspect ratio of such a rectangle is a rough compromise between the aspect ratio of a person viewed from the front and one viewed from the side with legs fully extended during a step. [2]

A sliding window moves across the image, computing a HOG descriptor at each position for classification.

Window size balances context, accuracy, and computation. Small windows capture finer detail but may miss full objects, while large windows include more context but are slower and can reduce localisation precision by adding background noise.

In practice, the window should fully contain the object with slight context; 64×128 is standard for pedestrians, while square windows can add unnecessary background. Smaller strides improve localisation but increase cost, while larger strides are faster but less precise. HOG is often paired with an image pyramid to detect objects at multiple scales, increasing computation [2].

5.5 Summary of Parameter Trade-offs

Parameter	Lower Value	Higher value
-----------	-------------	--------------

Cell size	More spatial detail but higher sensitivity to noise and small variations	More robust due to averaging, but reduced fine structural detail
Number of orientation bins	Smoother representation with less directional detail	More precise edge orientation encoding but more noise-sensitive and sparse
Block size	More local normalisation, preserving fine detail	Broader normalisation, improving robustness but reducing locality
Block overlap	Faster computation with lower robustness	Slower computation but improved accuracy due to redundancy and context
Window size	More local focus with less contextual information	More context but higher computational cost and potential background noise
Stride	More precise localisation but slower computation	Faster computation but less precise detection

Overall, HOG parameters balance detail, robustness, and computational cost. Standard choices like 8×8 cells, 9 bins, 2×2 overlapping blocks, and a 64×128 window offer a strong practical compromise, especially for pedestrian detection.

Effects of Extreme Choices

- Very small cells (e.g. 2×2): feature explosion and extreme sensitivity to small pixel shifts
- No block overlap: reduced robustness to illumination and shadows, leading to unstable detection
- Too many bins (e.g. 36): overfitting to precise edge angles and poor generalisation
- No normalisation: strong sensitivity to lighting conditions, breaking consistency across images
- Large window (e.g. 128×128 for pedestrians): increased background noise and reduced classification reliability

6. Failure Modes

HOG is powerful for capturing local edge structure, but it does not truly understand objects. It only encodes gradient distributions in fixed regions, so it can fail when shape, scale, visibility, or background vary significantly.

6.1 Rotation Sensitivity

A key limitation of HOG is that it is not rotation-invariant. Because it uses fixed spatial cells and orientation bins, rotating an object shifts its gradients across bins and cells, producing a different feature vector even when the object is unchanged.

As a result, a model trained on upright objects may fail when objects are rotated or viewed from unusual angles. For example, a pedestrian detector trained on upright people may miss someone lying sideways because expected vertical edge patterns no longer match.

6.2 Scale Sensitivity

HOG is also sensitive to scale because it uses a fixed detection window (e.g. 64×128). If an object is too small, key details are lost; if it is too large, it may not fit properly, breaking the expected spatial structure.

As object scale changes, gradient patterns no longer align with the expected cell and block structure, making descriptors inconsistent with training data. HOG addresses this using an image pyramid, resizing the image and applying the detector at multiple scales, but at a higher computational cost.

6.3 Occlusion and Articulation

HOG relies on consistent edge structure, so it is sensitive to occlusion and pose changes. If parts of an object are hidden, key gradients are lost or replaced by background edges, altering the descriptor and reducing detection accuracy.

HOG struggles with strong articulation because it assumes stable edge structure. While small movements are tolerated, large pose changes (e.g. sitting or crouching) shift gradients across cells and bins, producing feature vectors that differ from the learned template.

6.4 Background Clutter and the “HOGgles” Effect

HOG struggles in cluttered scenes because it relies only on gradient patterns, not semantic understanding. Background textures like branches, railings, or shadows can mimic object edges, leading to false positives. [5]

This is commonly seen in visualisations like “HOGgles,” where background textures can mimic learned object templates. For example, Vondrick et al. show a case where a detector produces a high-scoring car detection on sea water; although the original image clearly contains no car, the reconstructed HOG visualization appears car-like in feature space [5].

To reduce these errors, HOG systems use hard negative mining, adding false positives back into training as negatives so the classifier learns to better reject confusing background patterns.

6.5 Lighting Extremes

HOG is more robust to lighting changes than raw pixels due to gradients and block normalisation, but it still struggles under extreme lighting where gradient information is heavily degraded.

In very dark or overexposed scenes, object boundaries can disappear or become noisy, producing weak or unreliable gradients. Strong shadows may also create misleading edges. While block normalisation reduces brightness effects, it cannot recover lost structure, so HOG degrades under extreme lighting.

6.6 Lack of Colour and Texture Information

HOG encodes shape through edges and is largely colour- and texture-insensitive. This means objects with similar silhouettes (e.g. people, mannequins, posters) can produce very similar descriptors if their edge structure matches.

This makes HOG effective for objects with strong outlines but weak at distinguishing different objects with similar shapes. Modern deep learning methods overcome this by learning colour, texture, and semantic features, which is why CNNs have largely replaced HOG in many vision tasks.

6.7 Summary of HOG Limitations

Failure mode	Root cause	Intuition
Rotation sensitivity	Fixed orientation bins and spatial layout	Edge directions shift between bins, breaking the learned “signature” (barcode-like mismatch)
Scale sensitivity	Fixed detection window size	Object parts no longer align with expected cell/block structure
Occlusion	Missing or replaced spatial regions	Parts of the gradient “signature” disappear or are corrupted
Articulation	Rigid spatial template assumption	Limb movements relocate gradients into different cells, breaking structure
Background clutter	Gradient-based matching without semantics	Random textures can produce similar gradient patterns to real objects
Lighting extremes	Gradient degradation or distortion	Useful gradients disappear in darkness or are overwhelmed/altered by shadows and glare
No colour/texture information	Shape-only representation	Objects with similar silhouettes but different meaning become indistinguishable

Quick Reference

Key Concept

Shape over Details	HOG captures the overall shape and structural information of an object by tracking where pixel intensity changes sharply, discarding unstable details like exact colour or minor lighting fluctuations.
Gradients Capture Structure	Object boundaries and contours are highlighted by high rates of pixel intensity change, creating large gradient magnitudes. Flat uniform areas generate small gradients and provide less structural data.
Orientation	The direction of a gradient tells us how edges are oriented, creating distinctive patterns that identify specific shapes.
Invariance	Grouping pixels into cells provides spatial invariance, ensuring that small geometric shifts or noise do not drastically change the overall distribution of orientations in that area.

Equations / Functions

Magnitude	$\sqrt{Gx^2 + Gy^2}$
Orientation	$\arctan \frac{Gy}{Gx}$
L2-Norm	$v_{norm} = \frac{v}{\sqrt{\ v\ ^2 + \epsilon^2}}$

Parameter Ranges

Cell Size	Dalal and Triggs found that 6 to 8 pixels work best for human detection. Smaller cells capture finer structural detail but increase sensitivity to noise, while larger cells provide more robustness but lose detail [1].
Orientation Bins	The standard bin is 9 bins which covers 180 degrees. Fewer bins make the representation smoother but lose directions, while more than 9 bins encode edges precisely but there would be an increase in noise sensitivity.
Block Normalization	A common block size is 2x2 cells (which is 16x16 pixels). Smaller blocks preserve fine local contrast, while larger blocks increase robustness.

Overlapping blocks	Overlapping blocks improve accuracy by providing redundant context, but they require slower computation and produce larger feature vectors.
Window Size	This is a classic pedestrian detection window which is 64x128 pixels.

Glossary of Common Misconceptions

Rotation Sensitivity	A misconception is that HOG naturally handles rotated objects, but actually HOG is highly sensitive to rotation because it stores edge orientations in fixed spatial locations. For example if an object rotates then the descriptor changes drastically.
Scale Sensitivity	A misconception of Scale Sensitivity is that HOG automatically finds objects of any size. However, objects must roughly match the detection window size. To fix this, systems must heavily rely on an Image Pyramid, which repeats the sliding window at multiple sizes.
Background Clutter (HOGgles)	A misconception is that HOG understands what an object semantically is. But HOG only measures gradient patterns. Clutter can mimic shapes which can lead to false positives.
Lighting Extremes	A misconception is that Block normalisation fixes all lighting issues. Nevertheless, while it helps, extreme shadows can completely erase useful gradients.
Handling Complex Poses	A misconception is in which HOG easily handles partial hiding or flexible body movements. However, both occlusion and articulation rearrange expected edge patterns, causing the descriptor to fail.

Concise Definitions of Key Terminology

Feature detection	It creates a more robust representation of an image by extracting important visual information while discarding unstable details.
Bilinear interpolation	It is used to split a gradient's vote between the two nearest orientation bins to prevent aliasing and sudden jumps when a gradient angle shifts slightly.
Sliding window	A fixed size frame that moves across the image to evaluate HOG descriptors at each position.
L2-Hys normalisation	A technique that follows standard L2 normalisation but with an extra clipping and a renormalising step to prevent extremely strong edges from dominating the feature descriptor.
Hard negative mining	A training technique where false positives are intentionally added back into the negative training set so the classifier learns to reject confusing background patterns.

Key Papers

[1] Dalal and Triggs — Original HOG Paper

IEEE citation:

N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)*, San Diego, CA, USA, 2005, vol. 1, pp. 886–893.

URL:

<https://lear.inrialpes.fr/people/triggs/pubs/Dalal-cvpr05.pdf>

Summary:

This is the original paper that introduced HOG as a highly effective feature descriptor for human detection. Dalal and Triggs studied the problem of detecting pedestrians in images using locally normalized histograms of gradient orientations. The paper compares different design choices, including gradient filters, orientation binning, cell size, block size, block overlap, and normalization methods. Their experiments showed that dense grids of local gradient histograms, especially when normalized over overlapping blocks, significantly outperformed many previous handcrafted descriptors for pedestrian detection. The paper also established the classic HOG plus linear SVM framework.

Relevance:

This is the foundation paper for the project. It defines the HOG descriptor and justifies many standard parameter choices, including 9 unsigned orientation bins, local block normalization, and the 64x128 pedestrian detection window [1].

[2] Tomasi — Histograms of Oriented Gradients Notes

IEEE citation:

[2] C. Tomasi, "Histograms of Oriented Gradients," Duke University, Course Notes, 2015.

URL:

<https://courses.cs.duke.edu/fall15/cps274/notes/hog.pdf>

Summary:

Tomasi's notes provide a clear educational explanation of the HOG descriptor and its use in pedestrian detection. The notes explain how a detector slides a fixed-size window over an image pyramid, computes a feature vector for each window, and passes that feature to a classifier. They also describe the main HOG stages: gradient computation using central differences, cell orientation histograms, bilinear interpolation between orientation bins, block normalization, and final feature-vector construction. The notes are especially useful

because they explain the classic 128 x 64 pedestrian window and show how this setup leads to a 3780-entry HOG descriptor.

Relevance:

This source is directly relevant because it explains HOG in a step-by-step way, including sliding windows, image pyramids, cell histograms, block normalization, bilinear interpolation, and the final descriptor length. It supports the technical sections of the report and helps clarify how the original Dalal and Triggs method is implemented in practice [2].

[3] Viola and Jones — Real-Time Object Detection

IEEE citation:

P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR 2001)*, Kauai, HI, USA, 2001, vol. 1, pp. I-511–I-518.

URL:

<https://www.cs.cmu.edu/~efros/courses/LBMV07/Papers/viola-cvpr-01.pdf>

Summary:

Viola and Jones introduced one of the most influential classical object detection frameworks, especially for face detection. Their method used Haar-like features, integral images for fast feature computation, AdaBoost for selecting useful features, and a cascade classifier for efficient detection. Although it does not use HOG, it represents the same pre-deep-learning era of object detection, where handcrafted features were combined with machine learning classifiers. It also showed that sliding-window object detection could be made practical in real time if the features and classifier were designed efficiently.

Relevance:

This paper provides historical context for HOG. It shows what object detection looked like before CNNs and helps explain why HOG plus SVM was such an important development for shape-based detection [3].

[4] Lowe — SIFT Descriptor

IEEE citation:

D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, Nov. 2004.

URL:

<https://www.cs.ubc.ca/~lowe/papers/ijcv04.pdf>

Summary:

Lowe's SIFT paper introduced one of the most influential handcrafted feature descriptors in computer vision. SIFT detects distinctive keypoints and describes them using local gradient orientation histograms. Unlike HOG, which is usually computed densely across a fixed detection window, SIFT is usually computed around sparse keypoints and is designed to be robust to scale and rotation changes. The paper demonstrated reliable matching between different views of objects and scenes, even under changes in scale, rotation, noise, illumination, and viewpoint.

Relevance:

SIFT is useful for comparison because it also uses gradient orientation histograms. Comparing SIFT with HOG helps show that HOG is dense and detection-oriented, while SIFT is sparse, keypoint-based, and designed for matching [4].

[5] Vondrick et al. — HOGgles: Visualizing Object Detection Features

IEEE citation:

C. Vondrick, A. Khosla, T. Malisiewicz, and A. Torralba, "HOGgles: Visualizing object detection features," in *2013 IEEE International Conference on Computer Vision (ICCV)*, Sydney, NSW, Australia, 2013, pp. 1–8.

URL:

https://openaccess.thecvf.com/content_iccv_2013/papers/Vondrick_HOGgles_Visualizing_Object_2013_ICCV_paper.pdf

Summary:

Vondrick et al. introduced methods for visualizing what HOG-based object detectors "see." Since HOG descriptors are high-dimensional and difficult for humans to interpret directly, the paper proposed inverting HOG-like feature representations back into natural-looking images. This allowed researchers to inspect false positives and understand why detectors sometimes fail. A key finding was that some false detections may look clearly wrong in the original image but appear object-like in HOG feature space. This showed that certain errors are caused by limitations of the feature representation itself, not only by poor training or weak classifiers.

Relevance:

This paper is especially useful for the failure modes section. It supports the idea that background clutter can accidentally produce HOG descriptors that resemble real objects, causing false positives — the "HOGgles" effect [5]. The paper specifically discusses cases where false alarms look wrong to humans in image space but deceptively similar to true positives in HOG space [5].